CS781 Phase 1: Legal Language Model Pretraining

Evidence-based working report section, 9 November 2026. The final capstone report must use the course LaTeX template, cover all seven phases within its page limit, link the public code repository, and include a truthful LLM usage declaration. This section concerns Phase 1 only. Its prose and analysis were drafted with Codex assistance and require the student's review against the course's LLM-use rules before submission.

Task and data

Phase 1 trains a legal language model **from random initialization** on the provided Indian legal corpus. At evaluation, each input is a 512-word legal-text prompt and the output is exactly 128 new words, without repeating the prompt. The primary metric is BERTScore F1; ROUGE-L F1 and BLEU-4 are secondary. The four supplied training sources are Acts, High Court judgments, Supreme Court judgments, and Tribunal documents. The course describes more than 4.5 billion tokens across the complete corpus. It provides no public Phase 1 development set, so development and confirmation sets must be constructed from training data. Official test continuations remain private. The permitted model range is 50M–300M parameters, with no pretrained language-model weights or external commercial generation API.

The recovered run used a pinned revision of the official course dataset (**89edba6a44d6bb9593fd13b6895b5cd3b06eb82e**). The catalog held 2,608 Acts, 38,854 High Court, 3,165 Supreme Court, and 2,560 Tribunal documents. Exact normalized duplicates and near duplicates were clustered using SHA-256 and a 64-bit five-word-shingle SimHash before a seeded 98/1/1 train/dev/confirmation assignment; the intent was to keep related documents in one split. This procedure cannot prove complete semantic deduplication. Prepared data contained 189,729 training examples, 144 development prompts, and 127 separate confirmation prompts. The training examples per source were 46,792 Acts, 51,067 High Court, 46,681 Supreme Court, and 45,189 Tribunal. These are **example counts**, not corpus token counts.

Two training-example types were alternated by example count: exact 512-word prefix/128-word continuation windows and nonoverlapping token chunks with a randomly cut observed prefix. One quarter of documents, chosen deterministically by ID hash, used the full 8,192-token capacity for token chunks; the others used 2,047-token chunks. The loss was applied only to the continuation tokens. This mixed stream connects the official continuation format to longer-context language modeling, but it does not isolate a source-mixing or context-length effect experimentally.

Tokenizer and architecture

LegalSpan used a **handwritten byte-level BPE learner and encoder/decoder**, fitted only on the training split. The tokenizer fit read up to 25 million characters from each of the four sources, 100 million characters total, and produced a 32,768-token vocabulary. Byte fallback and reversible offsets let the implementation assign prompt-only role cues to tokens. These roles were simple surface heuristics: digits, statutory cues, capitalized name-like pieces, structural punctuation/newlines, and ordinary text. They were not gold legal-entity labels.

The model has **112,095,887 trainable parameters**. Its base is a 12-block causal Transformer with hidden width 768, 12 attention heads of width 64, 2,048-dimensional SwiGLU feed-forward layers, pre-RMSNorm, rotary positions, tied token/output embeddings, and an 8,192-token maximum sequence. Causal self-attention only reads the observed prefix at each position. The last four blocks add a 128-dimensional prompt route. Each route projects generation queries and prompt states to query/key/value vectors, scores only eligible observed prompt positions, adds a learned role-dependent bias and a bounded recent-token continuity feature (up to four observed tokens), and computes a weighted prompt context. A learned sigmoid gate scales the route update before it is added to the generation-side block state. The earlier prompt states are left unchanged, preventing future-prompt information from entering the route memory.

At the output layer, a second learned gate mixes the ordinary vocabulary distribution with a prompt-copy distribution. The latter accumulates attention mass from eligible prompt positions onto their token IDs. In notation, for next token y, P(y) = (1-g) P_vocabulary(y) + g P_copy(y). Training uses the normalized mixture likelihood, not a separate unnormalized copy bonus. The design tests whether prompt role cues and observed local continuity help preserve legal terms and citations. Copying is an established idea; the claim here is a tested course-specific combination, not invention of pointer generation.

Training, selection, and generation

The archived **run.json** records random initialization, no synthetic smoke data, AdamW with learning rate 3 x 10^-4, betas (0.9, 0.95), weight decay 0.1, 2% linear warm-up followed by cosine decay to 10% of peak rate, gradient clipping at 1.0, BF16 autocast, gradient checkpointing, microbatch size one, and 32 accumulated examples per optimizer update. The full run completed **11,888 optimizer steps** and **209,927,459 supervised target tokens** against a 209,914,685-token target; 581,555,637 input tokens were processed in about 21,988 seconds (6.11 hours). Input-token and supervised-target-token counts measure different things. Training loss in the step log is not a held-out perplexity measurement.

Checkpoints were saved during training and at intermediate token milestones. The pipeline evaluated several saved candidates and three decoding policies on the development split, selected by development BERTScore F1, and then evaluated the frozen choice on confirmation data. The selected policy was nucleus sampling with temperature 0.8, top-p 0.9, repetition penalty 1.05, and a 2,048-new-token cap. Generation stopped only after 128 complete words; EOS was barred before that point. The submitted output ZIP held a single root-level **prediction.json** with 1,000 exact official test IDs and 128 words per generated continuation. Archived validation of the ZIP contract does not itself prove a leaderboard score.

Results and controlled comparisons

Phase 1 lineage	Distinct computation	Historical Codabench BERTScore F1
A0	Dense 12-block causal decoder	0.8304
B0	6-block encoder / 6-block cross-attending decoder	0.7972
C	10-block prompt encoder / 4-layer GRU generator	0.8237
Improved A / LegalSpan	A decoder plus routed prompt memory and copy mixture	**0.8341**
Improved B	B0 graph with continued task-aligned training	0.8244

These are **historical screenshot-reported Codabench values** recorded in the October 6 archive inventory, not fresh leaderboard retrieval. LegalSpan was the highest evidenced Phase 1 Codabench submission, 0.0037 absolute above A0. B1 and C1 own-BPE reruns have valid prediction ZIPs but no independently verified Codabench score in the preserved record. D222 was built after the Phase 1 deadline and has a later local benchmark only, so it is not counted as an on-time Phase 1 leaderboard result. The October 6 records show three broad model families (A, B, C) and a substantially altered LegalSpan computation graph; whether this meets the course's four-architecture rule remains for course staff to decide.

The matched LegalSpan runs shared the own BPE/data stream and approximately the same supervised-token budget. Their selected local metrics were:

Route variant	Dev BERTScore F1 (n=144)	Confirmation BERTScore F1 (n=127)	Dev ROUGE-L F1	Dev BLEU-4
Dense control	0.865228	0.865052	0.347439	0.262007
Generic prompt route	**0.868770**	0.865343	**0.358181**	**0.270168**
Full role/continuity route	0.867648	**0.869258**	0.350857	0.264881

The generic route exceeded the full route on the small development set by 0.001122 BERTScore F1; the full route exceeded it on the separate confirmation set by 0.003915. A paired 5,000-resample bootstrap over archived per-example BERTScore values gave a 95% interval of [-0.004886, 0.002749] for full minus generic on development data and [-0.001572, 0.009617] on confirmation data. Both include zero. Full minus dense control on confirmation data was 0.004206, interval [0.000496, 0.008100]. These are exploratory uncertainty summaries, not proof that the routing mechanism always improves generation. The small splits and multiple policy/checkpoint comparisons limit generalization.

The selected full model's local development metrics were BERTScore F1 **0.867648**, ROUGE-L F1 **0.350857**, and BLEU-4 **0.264881**. On the separate confirmation set they were **0.869258**, **0.368787**, and **0.287066**. Local held-out metrics and the private Codabench value come from different evaluation sets and must not be compared as if they used identical cases.

Interpretation and limits

The course's requested exploration covered tokenizer design, model size, four-source data, context, optimization, decoding, validation, and checkpoint selection. The recovered artifacts provide direct evidence for the own BPE, 112.1M-parameter graph, deterministic data split, mixed 512/128 and long token windows, 8,192-token capacity, training schedule, several saved checkpoints, three decoding policies, and controlled route variants. They do **not** establish a full factorial study of tokenizer sizes, data-source ratios, context lengths, learning-rate schedules, or compression. No validation-loss/perplexity curve is preserved as a basis for a quantitative loss-versus-generation claim. Token-level training loss and generated-text BERTScore measure different objectives. The higher local score of the generic route on development data is a meaningful negative result for the full-role hypothesis, even though the final full model did better on confirmation and had the strongest evidenced Phase 1 Codabench score.

The 128-word contract is exact, but BERTScore does not verify legal factuality or citation correctness. The role cues can misclassify headings or capitals, and copied words can preserve surface form while still being used in the wrong legal context. The measured improvement over A0 on Codabench is small; score alone does not establish causal benefit from any one component. A next controlled experiment would evaluate independent role, continuity, and copy toggles on more held-out documents, using citation and factuality checks alongside BERTScore.

Reproducibility and provenance

The best model's recovered Python modules, config, run entry points, and companion review notebook are in the Phase 1 code package. The notebook was assembled **after** the GPU training run from archived source and displays archived run evidence; it is not an original executed training notebook. The original October 6 archive preserves checkpoint weights, tokenizer, prepared data, logs, run/evaluation JSON, and the submitted prediction ZIP. The code package deliberately excludes large weights and course data. Source and artifact hashes are recorded in the archive manifests. Original shell scripts include paths specific to the former remote workstation and must be adapted to a new environment; a fresh reproduction was not performed for this report.

The Phase 1 code and documentation repository is https://github.com/k27321278-a11y/cs781-capstone-code. The final all-phase ZIP must follow the announced name and place **code/** and **report/** directly at its root. This Phase 1 package supplies only Phase 1 materials and is not the final seven-phase submission. The course report template also requires a global LLM Usage Declaration; the true extent of Codex assistance, including this draft and earlier design/code assistance, must be stated rather than omitted.

Sources and attribution

- CS781 updated Capstone Document, Phase 1 §§7.1–7.9 and final report instructions: https://docs.google.com/document/d/1olqoT5KPhf5CSJLBNjXGyKyyyQDBr6jcnWqPUh5cNk4/edit
- Preserved **CS781_Improved_A_LegalSpan_Complete_20261006.zip**: **artifacts/run.json**, **training_status.json**, **selection.json**, **evaluation/confirmation.metrics.json**, route ablation records, prepared-data manifest, tokenizer metadata, and source code.
- October 6 archive inventory: **CS781_ARCHIVE_INVENTORY_20261006.md**, including historical Codabench screenshot attribution and archive hashes.
- Sennrich et al. (2016), byte-pair encoding: https://aclanthology.org/P16-1162/
- Touvron et al. (2023), LLaMA backbone elements: https://arxiv.org/abs/2302.13971
- See et al. (2017), pointer-generator mixture: https://aclanthology.org/P17-1099/
- Gu et al. (2016), CopyNet: https://aclanthology.org/P16-1154/